

Faculty of Computing

SE1020 – Object Oriented Programming

Year 1 Semester 2 (2025)

Tutorial 04

Question 01

In a Smartphone, the Battery and Processor are essential components that cannot function independently outside the device. The Smartphone fully owns and controls these components, meaning their existence depends on the Smartphone. If the Smartphone is destroyed, the Battery and Processor are also destroyed.

- a) Create a class called **Battery** with the attribute capacity (int) as an attribute and implement a method to display battery details.
- b) Create another class called **Processor** with model (String) as an attribute and implement a method to display processor details.
- c) Implement a class called **Smartphone** that contains a Battery and a Processor.
 - i. Initialize Battery and a Processor instances inside the Smartphone constructor.
 - ii. Provide a method to display full smartphone details, including battery and processor information
- d) Create a class called **SmartApp** with the main method to:
 - i. Create an object of Smartphone (with a Battery and Processor).
 - ii. Display all details.

Sample Output:

```
Smartphone Brand: Samsung Galaxy
Battery Capacity: 5000mAh
Processor Model: Snapdragon 888
```

Question 02

A Car is a type of Vehicle, meaning it inherits common properties of a Vehicle such as brand and speed. At the same time, a Car is composed of an Engine, which means the Engine is a necessary component that cannot exist independently outside the Car.

- a) Create a class called **Vehicle** with brand (String), and speed (int) as attributes. Then implement a method to display vehicle details.
- b) Create a **Car** class that extends Vehicle and contains model (String) as an additional attribute.
 - i. Implement a parameterized constructor to initialize all the attributes.
 - ii. Create a method to display car details.
- c) Implement a class called **Engine** with engineType (String) as an attribute. Then implement a method to display engine details.
- d) As the Engine is a part of the Car and Engine cannot exist without a Car, modify the Car class to include an Engine instance. Initialize Engine inside the Car's constructor.
- e) Modify the display method to display full car details, including engine information.
- f) Create a class called **CarApp** with a main method to:
 - i. Create a Car object (which includes an Engine).
 - ii. Display all details.

Question 03

A University has multiple Professors who specialize in different subjects. Each Professor exists independently and is not owned by the University. If the University closes, the Professors can continue to work elsewhere. The University should store multiple Professor objects using a simple array and allow displaying all associated details.

- a) Write a Java program to create a class called **Professor** with attributes for name and subject, along with a method to display professor details.
- b) Implement a class called **University** that stores multiple Professor objects using a simple array and provides a method to display all university details, including the associated professors.
- c) Create a main method in a **UniversityApp** class to create multiple Professor objects separately, assign them to a University, and display all details.

Sample Output:

```
University: Tech University
Professors:
Professor Name: Dr. Smith, Subject: Computer Science
Professor Name: Dr. Johnson, Subject: Mathematics
Professor Name: Dr. James, Subject: Science
```

Question 04

A Transport Management System manages different types of vehicles used in a transportation company. Each vehicle has a brand and speed, but there are specialized vehicles such as Buses and Trucks that have unique features.

Each Vehicle contains an Engine that determines its power, meaning Engine is a part of the Vehicle and cannot exist without it.

Additionally, the transportation company owns multiple Drivers, and each Driver is associated with a Vehicle. However, a Driver can exist independently of a specific vehicle.

Different types of vehicles have unique implementations of the start() method, which need to be overridden in the respective classes

- a) Implement a class called **Vehicle** with brand(String) and speed(double) attributes.
 - i. Create a method called start(), that prints a generic message.
- b) Create two classes called **Bus** and **Truck** as subclasses of Vehicle.
 - i. Override the start() method to display unique messages for each type.
- c) Implement an **Engine** class with engineType(String) as an attribute.
 - i. The Engine should be part of a Vehicle (Composition) and initialized within the Vehicle constructor.
- d) Create a class called **Driver** with name(String) and licenseNumber(String) attributes.
 - i. A Driver should be associated with a Vehicle using Aggregation
- e) Modify the **Bus** and **Truck** classes to include a Driver for each class(Aggregation).
- f) Create a **TransportApp** class with a main method to:
 - i. Create objects for different Vehicles (Bus, Truck) with their Engines.
 - ii. Assign Drivers to those Vehicles.
 - iii. Start the vehicles and display full details, including the associated Driver and Engine.

Sample Output:

--- Bus Details ---

Bus is starting... Passengers are boarding.

Vehicle Brand: Volvo

Speed: 80.0 km/h

Engine Type: Diesel

Driver Name: John Doe, License: DL12345

Passenger Capacity: 50

--- Truck Details ---

Truck is starting... Loading goods.

Vehicle Brand: Mercedes

Speed: 60.0 km/h

Engine Type: Turbo Diesel

Driver Name: Alice Smith, License: DL67890

Load Capacity: 10.0 tons
